Homework 4 **Author: Vasudev Prasad Bethamcherla**
Author: Piter Garcia Due Date: **Monday, 7/1**

1. Let n be the integer in Computer Problem 6.9.4 (page 198):
n = 618240007109027021

- **Give the prime factor p of n such that p-1 has only small prime factors.**

To answer this question, by building a program with help of the java tools, we preceded the follow steps:

First, we use the BigInteger properties from java to accomplish large number computations such as: mod, power, gcd, methods that we have previously wrote in our previous assignment and are of great use for the factorization of a prime number.

Second, we use the Miller Rabin algorithm program that we built from the previous assignment. This code was of great used to figure out whether or not p was a prime number, in order to find an easier implementation.

Third, we built the methods bigN(n), p_1Factor(n,i), and cactor(n) to obtain the following behaviors. bigN(n), factors n and displays its p, first and second value, along with the last b value used to factor p-1. The p_1Factor(n,i), takes the value of p-1(n in this case) to factor it. To do so it generates several b values, where b = k^n mod n and 1 < k < i, n and its range is from 1 to i. The factor (n), is the final method which takes care of the basic factorization rules. If p = 1 or p=n is a failure and therefore it returns, values of p are assigned in this method for display purpose.

To have a better understanding of the program, here are some calculations that illustrate how our programs function and output.

For n = 6182400071092721

```
P - 1: 250387201
True -> means that p is a prime number
P - 1: 2469135821 -> obtained by dividing n/p
True -> means that p is a prime number

Total Factorization of P: 250387201
Total Factorization of P: 2469135821

n = pq: 618240007109027021

Small Prime Factors of p = 250387201
2^8 3^5 5^2 7^1 23^1

Small Prime Factors of p = 2469135821
2^2 5^1 123456791^1
```

Homework 4 **Author: Vasudev Prasad Bethamcherla**
Author: Piter Garcia Due Date: **Monday, 7/1**

Source Code: Reference can be better explain by running PrimeFactor.java

```
public class PrimeFactor {

   private static BigInteger pf = null;
   private static BigInteger pft = null;

   public static void main(String[] args){
         BigInteger n = new
         BigInteger("61824000710902721");
         bigN(n);

         long l = 100;
         f prime = new f(pf, l);
         System.out.println("n: " + (pf));
         System.out.println("P - 1: " + pf);
         System.out.println(prime.isPrime());

         f prime1 = new f(pft, l);
         System.out.println("P - 1: " + pft);
         System.out.println(prime1.isPrime());

         if( prime.isPrime() &&
         prime1.isPrime()){
         System.out.println("Total Factorization
         of P: " + (pf));
         System.out.println("Total Factorization
         of P: " + n.divide(pf));
         System.out.println("n = pq: " +
         (pf).multiply(pft) + "\n");
         }
   }

   public static void bigN(BigInteger n2){
         factor(n2);

         int i = 1;
         int a = 2;
         BigInteger b = new BigInteger("1");
   }

   private static void factor(BigInteger n) {
         BigInteger pf = p_1Factor(n, 2000);
      if( pf.equals(n) || pf.equals(BigInteger.ONE))
         return;

      BigInteger pft = n.divide(pf);
      PrimeFactor.pf = pf;
      PrimeFactor.pft = pft;
   }


public static BigInteger gcd(BigInteger a, BigInteger n2){

      //      System.out.println(" number is : " + a);
      while( !a.equals(n2)){

         if( a.compareTo(n2) == 1)
               a = a.subtract(n2);
         else
               n2 = n2.subtract(a);

      }
      return a;

   }

   public static BigInteger p_1Factor(BigInteger n, int
bound){
      BigInteger a = new BigInteger("2");

         BigInteger btemp;
         BigInteger b = a.mod(n);
         for( int i=1; i <= bound; i++){
               btemp = new BigInteger(i+"");
               b = b.modPow(btemp, n);
               //          System.out.println(b);
         }

      return b.subtract(BigInteger.ONE).gcd(n);
   }
}
```

- **Give the prime factorization of p-1.**

To obtain the factorization of p-1, the same approach as above was used. However, this time we used n = p, which in this case is, was equal to 250387201 and or 2469135821. We proceeded to run the program for both values and compare the result. Below is an output example of the program built which confirms the following:

Example output:

```
n: 250387201
P - 1: 250387201
true
P - 1: 2469135821
true
Total Factorization of P:
250387201
Total Factorization of P:
2469135821
n = pq: 618240007109027021
```

```
n: 250387201
P - 1: 250387201
true
P - 1: 2469135821
true
Total Factorization of P:
250387201
Total Factorization of P:
2469135821
n = pq: 618240007109027021
```

As we can see, the factorization of p is the number itself. This confirms the fact that phi (p) = p -1 when p is a prime. Therefore, since p is an odd prime number in this case the factorization number results being the number itself. This is truth for both of the values we obtained when factoring n, 250387201 and 2469135821.

Source Code:

```
BigInteger n2 = new BigInteger("250387201");
bigN(n2);
l = 100;
prime = new f(pf, l);
System.out.println("n: " + (pf));
System.out.println("P - 1: " + pf);
System.out.println(prime.isPrime());

prime1 = new f(pft, l);
System.out.println("P - 1: " + pft);
System.out.println(prime1.isPrime());                    if(
prime.isPrime() && prime1.isPrime()){
System.out.println("Total Factorization of P: " + (pf));
System.out.println("Total Factorization of P: " + n.divide(pf));
System.out.println("n = pq: " + (pf).multiply(pft) + "\n");
}
BigInteger n3 = new BigInteger("2469135821");
```

```
    bigN(n3);
l = 100;
prime = new f(pf, l);
System.out.println("n: " + (pf));
System.out.println("P - 1: " + pf);
System.out.println(prime.isPrime());

prime1 = new f(pft, l);
System.out.println("P - 1: " + pft);
System.out.println(prime1.isPrime());

if( prime.isPrime() && prime1.isPrime()){
System.out.println("Total Factorization of P: " + (pf));
System.out.println("Total Factorization of P: " +
n.divide(pf));
System.out.println("n = pq: " + (pf).multiply(pft) + "\n");
        }
```

- **Give the smallest value B_0 such that p - 1 | B_0!.**

In this question we made use of the code above code, since they are all related, and kept track of all the B values used to be able to compare them.

First Colum is B values, Second Colum is 2^B mod n values, and third column is p-1/B
Output Reference:

```
1 3  83462400
2 9  27820800
6 729  343466
24 282429536481  0
120 224607963196757581  0
720 274421885737827541  0
5040 369913000283081781  0
40320 171268397058352721  0
362880 23605649206140181  0
3628800 88747939183300361  0
39916800 35924790054016381  0
479001600 355379315439754281  0
6227020800 573420441450462421  0
87178291200 163620581610298441  0
1307674368000 84543568020296581  0
20922789888000 330280360978322821  0
355687428096000 222301648769207921  0
6402373705728000 134595784703342901  0
121645100408832000 303044881263419501  0
…
```

From this output we assumed, by testing, that 23 is the smallest non-trivial value since it does not form part of the smallest factors of n such as; 2^8 3^5 5^2 7^1 23^1.

Source Code: For a better reference see and run code of the first question.

```
public static void bigN(BigInteger n2){

    factor(n2);

    int i = 1;
    int a = 3;
    BigInteger b = new BigInteger("1");

    do{
       b = b.multiply(BigInteger.valueOf(i));
       System.out.print("  " + b + " ");
       System.out.print(BigInteger.valueOf(a).modPo
       w(b, n2.subtract(BigInteger.ONE)));
       System.out.println("  " +
       pf.divide(BigInteger.valueOf(a).modPow(b,
       n2.subtract(BigInteger.ONE))));
            i ++;

            }while(i!= 35);
    }
```

- **Run the Pollard p-1 algorithm (use the version from the slides) with B = B_0 and verify that it gives a non-trivial factor of n. Give this non-trivial factor of n.**

The entire code was explained and reference in part of one of this question. However, we can explain in detail how our Pollard p-1 method works in our program as well as its function in reference to the slides provided by the instructor.

```
public static BigInteger p_1Factor(BigInteger
n, int bound){
        BigInteger a = new BigInteger("3");

        BigInteger btemp;
        BigInteger b = a.mod(n);
        for( int i=1; i <= bound; i++){
                btemp = new BigInteger(i+"");
                b = b.modPow(btemp, n);
        //System.out.println(b);
        }
return b.subtract(BigInteger.ONE).gcd(n);
}
```

```
private static void factor(BigInteger n) {
    BigInteger pf = p_1Factor(n, 2000);
    if( pf.equals(n) || pf.equals(BigInteger.ONE))
            return;

    BigInteger pft = n.divide(pf);
    PrimeFactor.pf = pf;
    PrimeFactor.pft = pft;}
```

In the first method p_1Factor, bound contains the bound number for B and a is the first value for b, where b is equal to b^i and i < bound and i > 0. After all iterations have been performed, that is from i=1 till i = bound, we find the gcd value of b-1 that is gcd(b-1, n).

Second method, takes n where n is equal to p-1. After this p-1 is processed by the firs method p_1Factor, this output is checked not to be 1 or n, which is p-1 since this are the basic failure cases. Afterwards, we obtain the second prime factor number by dividing n original given number, not p-1, by the prime factor number obtained.

This approach was done on reference to the slides provided, and this was how we obtained the following numbers P - 1: 250387201 and P - 1: 2469135821.

Output Reference: for a better output reference see above output reference.

First Colum is B values, Second Colum is 2^B mod n values, and third column is p-1/B

```
1 2  125193600
2 4  62596800
6 64  3912300
24 16777216  14
120 127609531220500916  0
```

Source Code: Reference from the given code provided in the first question.

- **Give the largest value for B < B_0 such that the p-1 algorithm fails.**

The larges number below 23 that fails is 22.

- **Give a value for B > B_0 such that the p-1 algorithm fails and explain why it fails.**

The largest number for which B fails is B = n, and the reason why can be explained as follows:

If p is a factor of n, the integer we are aiming for, the p<= gcd(x – y, n) <= n since p divides both x – y and n. Let x be the current state of one sequence and y be the current state of the other. The GCD of |xy| and n is taken at each step. If this GCD ever comes to n, then the algorithm terminates with failure, since this means x = y and therefore the sequence has cycled and continuing any further would only be repeating previous work.

Therefore, observing output reference below, and taking into account that I tested it with i up to 100,000, any number after 24 will just keep on repeating previous work. Having said that, picking a value grater than B and that fails according the above description, B = 618240007109027021 is a value greater than B0 = 23 and it fails since it only repeats the previous work like the other thousands of B values after it.

Output Reference:

```
1 3  83462400
2 9  27820800
6 729  343466
24 282429536481  0
120 224607963196757581  0
720 274421885737827541  0
5040 369913000283081781  0
```

Homework 4 **Author: Vasudev Prasad Bethamcherla**
Author: Piter Garcia Due Date: **Monday, 7/1**

2.Exercise 6.8.12 (page 193):
You are trying to factor n = 642401. Suppose you discover that
$516107^2 = 7 \pmod n$
and $187722^2 = 2^2 * 7 \pmod n$
Use this information to factor n.

Work:

(516107 . 187722)2 = 22 . 72 (mod 612401).
(289038)2 = (14)2 (mod 612401).
gcd(289038 − 14, 642401) = 1129.

Output:

```
For e1 = 516107
For e2 = 187722
For e1 * e2 = 289038
2^2 * 7^2 =  2^2 * 7^2 = 14
GCD(289038 - 14, 642401) = 1129
Second Factor: 569
```

Source Code:

```
public static Integer mod(Integer a, Integer b){
        Integer temp = 0;

        temp = a/b;
        temp *= b;
        temp = a - temp;

        return temp;
    }
```

```
public static int gcd(int a, int b){
    while( a != b){

            if( a > b)
                a -= b;
            else
                b -= a;
    }
    return a;

}
```

Work Explanation:

We know that x != +- y such that x^2 = y^2 mod n then gcd(x-y, n) is a factor of n which is different from 1 and n, therefore, we are able to determine a non-trivial factor of n. We notice that we can multiply 516107 and 187722 mod n and still keep the square properties, and the multiple squared of both values is equal to 2*7 mod n, again keeping the square property of each number. Consequently, we can take x = 289038and y = 14 and obtain the factor number 1129, to obtain the second factor we just divide n by the resulting value 642401/1129 = 569.

Homework 4 **Author: Vasudev Prasad Bethamcherla**
Author: Piter Garcia Due Date: **Monday, 7/1**

3. When generating parameters for RSA we should take care to ensure q-p is not too small, where n = pq and q > p.

- **Suppose that q-p = 2d > 0, and n =pq. Prove that $n+d^2$ is a perfect square.**

Using the squaring binomial formula we can come up with the following formulation:
n + d^2 = pq + (q – p/2)^2 = pq + q^2/4 + p^2/4 – pq/2 = q^2/4 + p^2/4 + pq/2
n + d^2 = (p + q/2)^2 .

Solving for d we get:
d= (q-p)/2

Proving square:
$n+d^2$
$= pq + ((q-p)/2)^2$
$= (4pq + (q^2 -2pq + p^2))/4$
$= (q^2+2pq+p^2)/4$
$= ((q+p)/2)^2$

(p+q)/2 is in Z, therefore $n+d^2$ is a perfect square.

- **Given an integer n which is the product of two odd primes, and given a small positive integer d such that $n + d^2$ is a perfect square, show how this information can be used to factor n.**

From the work we just proved above d > 0 is (q − p)/2. However, since the primes p, q must be close then the difference q − p = 2d must be small, where q > p as shown below:

n = ((q + p)/2)2 − d2
Let x be an integer = (p + q)/2, which can only be slightly larger than √n, and x^2 − n is a square in Z.

We can try the following to come up with an integer x whose value is a square in Z.
x = ⌈√n⌉ + i, i = 0, 1, 2, . . . till x becomes a square in Z. Then q = x + d and p = x – d, and since n = pq with q > p from the equation n = ((q + p)/2)2 − d2. We prove that this satisfies the following mathematical expression below:

n + d^2 = m^2
n = m^2 – d^2
n = (m + d)(m – d)
p = (m + d) & q = (m - d)

○ **Use this technique to factor 2189284635403183.**

From the work above we find that d = $\sqrt{x^2 - n}$
Now, we first proceed to find the square root of the given value n. $\sqrt{n}$ = $\sqrt{2189284635403183}$ ≈ 46789791.99…
Second, we choose a value i, in this case we chose i = 0 so x ≈ 46789792, to obtain the following values by substituting the above equation:
d = $\sqrt{x^2 - n}$ = $\sqrt{467897922 - 2189284635403183}$ = 9 = 3^2.

Finally, we find √n ≈ 46789791.99 . . ., so trying k = 0, we find u = 46789792 and
d = pu2 − n = p467897922 − 2189284635403183 = 9 = 32, this is a square in Z.

From the above values, we get that p = 46789801 and q = 46789783. However, to be certain about our answers we checked to be if they were prime we our Miller Rabin algorithm and this was the answer we obtained.

We just reused the code used in the above questions, and to better explain n = p -1 = p in the first case, and in the second case n = p -1 = q. As we can see, they both are prime "true", this proves our approach was correct.

Output:

First Case:
```
n: 46789783
P - 1: 46789801
true
```

```
Second Case:
n: 46789783
P - 1: 46789801
true
```

Source Code: same as used the above questions.